

Contents

Hinweise, Taktik & Lösungsgedanken zu den Backlog-Storys	1
Erst mal die Karte verstehen	1
Epic 1 — Bewegung & Überleben	2
Epic 2 — Angriff	5
Epic 3 — Strategie & Zustände	6
Epic 4 — Qualität (optional)	7
Epic 5 — Turniervorbereitung	8
Wenn ihr komplett feststeckt	8

Hinweise, Taktik & Lösungsgedanken zu den Backlog-Storys

Diese Datei ist eure **Denkhilfe** für die Aufgaben aus [backlog.md](#). Sie gibt **keine fertige Lösung** vor, sondern hilft euch, selbst auf den Lösungsweg zu kommen: mit Bildern, Beispielen und Leitfragen.

Lest vorher unbedingt einmal den [Framework-Guide](#) — dort steht, was `sensors` enthält und welche `Actions` es gibt. Wenn ihr an einer Story hängt, sucht sie hier und arbeitet die Leitfragen der Reihe nach ab.

Erst mal die Karte verstehen

Alles dreht sich um Positionen auf einem 10×10-Raster. Wer die Koordinaten nicht sicher im Kopf hat, verrechnet sich bei **jeder** Story. Also zuerst das hier verinnerlichen:

Die drei Dinge, die euch am häufigsten stolpern lassen:

1. **(0, 0) ist oben links.** Nicht unten links wie im Mathe-Unterricht.
2. **y wächst nach unten.** NORTH (nach oben) heißt y wird **kleiner**, SOUTH heißt y wird **größer**. Das fühlt sich falsch an — ist aber so.
3. **x ist die Spalte (links/rechts), y die Zeile (oben/unten).**

Kleiner Merksatz für die Richtungen:

NORTH	= y - 1	(nach oben)
WEST	= x - 1	EAST = x + 1
SOUTH	= y + 1	(nach unten)

Die zwei wichtigsten Rechnungen

Fast jede Story braucht eine von diesen beiden. Schreibt sie euch einmal auf, dann könnt ihr sie überall wiederverwenden.

“Wie weit ist der Gegner weg?” — Abstand (Manhattan-Distanz):

```
import kotlin.math.abs
```

```
val abstand = abs(gegner.position.x - self.position.x) +  
              abs(gegner.position.y - self.position.y)
```

Das ist die Summe aus “wie viele Schritte nach links/rechts” **plus** “wie viele nach oben/unten”. Genau passend, weil man sich nur gerade (nicht diagonal) bewegen kann.

“In welche Richtung liegt der Gegner?” — Richtungsvektor dx/dy:

```

val dx = gegner.position.x - self.position.x // + = Gegner ist rechts, - = links
val dy = gegner.position.y - self.position.y // + = Gegner ist unten, - = oben

```

dx und dy sagen euch, wo der Gegner **relativ zu euch** steht. Fast die gesamte Bewegungs- und Schuss-Logik im ganzen Backlog ist am Ende nur: *schau dir dx und dy an und leite daraus eine Richtung ab.*

Wichtige Falle beim Vorzeichen: Wollt ihr **zum** Gegner hin, bewegt ihr euch in Richtung von dx/dy. Wollt ihr **weg** (fliehen), dreht ihr das Vorzeichen um. Das ist DIE Stelle, an der sich alle mal vertun — bei jeder Bewegungs-Story kurz gegenprüfen: “laufe ich gerade hin oder weg?”

Epic 1 — Bewegung & Überleben

Story 1.1 — Zufällige Bewegung · 1 Punkt

Ziel: Bot bewegt sich jeden Tick in eine zufällige Richtung. Reiner “Läuft mein Bot überhaupt?”-Test.

Leitfragen: - Wie bekomme ich alle vier Richtungen als Liste? (Tipp: `Direction.entries`) - Wie wähle ich davon eine zufällig aus? (Tipp: `.random()`) - Was muss `decide()` am Ende zurückgeben? Eine `Action.Move(...)`.

Taktik-Gedanke: Strategisch bringt Zufallslauf nichts — aber ihr seht sofort, ob euer Bot in der App auftaucht und sich bewegt. Genau darum geht’s hier. Erst wenn das klappt, lohnen sich die schwereren Storys.

Häufiger Fehler: `Action.Move` ohne Richtung, oder `Direction.random()` statt `Direction.entries.random()`.

Story 1.2 — Am Rand bleiben / Zentrum meiden · 2 Punkte

Ziel: Bot soll sich nicht in der Mitte aufhalten, sondern Richtung Rand.

Warum ist das taktisch klug? In der Mitte kann euch aus **allen vier** Richtungen jemand treffen. Am Rand fällt schon eine Richtung weg, in der Ecke sogar zwei:

Mitte (schlecht):	Rand (besser):	Ecke (am sichersten):
<pre> ↑ ← ● → 4 Angriffs- ↓ seiten </pre>	<pre> ↑ ← ● → 3 Seiten ↓ (Wand unten) </pre>	<pre> ↑ ● → nur 2 Seiten ↓ offen </pre>

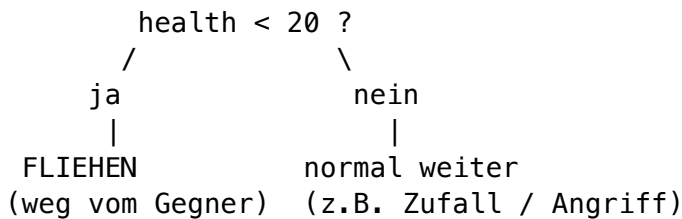
Leitfragen: - Wo ist die Mitte? `sensors.arenaWidth / 2` und `sensors.arenaHeight / 2`. - Wie weit bin ich von der Mitte weg? Mit `abs(pos.x - mitteX)`. - Wenn ich zu nah dran bin: in welche Richtung ist der Rand näher — links oder rechts? Vergleicht `pos.x` mit `mitteX`.

Taktik-Gedanke: Ihr müsst es nicht perfekt machen. Es reicht schon, wenn ihr **eine** Achse betrachtet: “Stehe ich links von der Mitte? -> weiter nach WEST. Sonst -> nach EAST.” Wer mag, macht dasselbe zusätzlich für oben/unten.

Story 1.3 — Flucht bei niedriger Gesundheit · 3 Punkte

Ziel: Sinkt `health` unter 20, läuft der Bot **weg** vom nächsten Gegner. Sonst verhält er sich normal (z.B. wie in Epic 2, oder erst mal Zufallslauf).

Das ist der erste “wenn ... dann anders”-Bot. Bisher hat euer Bot immer dasselbe gemacht. Jetzt baut ihr eine **Weiche**: je nach Situation zwei ganz verschiedene Verhalten. Genau dieses Muster braucht ihr später in Story 3.1 (Zustandsmaschine) wieder — nur mit mehr Zuständen.



Das Akzeptanzkriterium verlangt drei Dinge — hakt sie einzeln ab: 1. `health < 20` -> weg vom **nächsten** Gegner bewegen. 2. `health >= 20` -> normales Verhalten (irgendwas Sinnvolles, kein Absturz). 3. **kein** Gegner mehr da -> keine Exception, sondern `Action.Wait`.

Schritt 1 — nächsten Gegner finden (und den Leer-Fall abfangen) Fliehen könnt ihr nur **vor** jemandem. Also zuerst: wer ist der nächste Gegner? Der Leer-Fall ist keine Kür, sondern Pflicht (Kriterium 3) — ohne ihn stürzt der Bot ab, sobald der letzte Gegner tot ist und `sensors.others` leer wird.

```
import kotlin.math.abs
```

```
val self = sensors.self.position
```

```
// Kein Gegner mehr da? Dann sofort raus, nicht weiterrechnen.
```

```
if (sensors.others.isEmpty()) {
    return Action.Wait
}
```

```
// Nächsten Gegner mit einer Schleife suchen (den mit dem kleinsten Abstand)
```

```
var naechster = sensors.others[0]
var kleinstAbstand = abs(naechster.position.x - self.x) + abs(naechster.position.y - self.y)
for (gegner in sensors.others) {
    val abstand = abs(gegner.position.x - self.x) + abs(gegner.position.y - self.y)
    if (abstand < kleinstAbstand) {
        kleinstAbstand = abstand
        naechster = gegner
    }
}
```

So funktioniert die Schleife: Wir merken uns anfangs den **ersten** Gegner als “nächsten”. Dann gehen wir alle Gegner durch und rechnen für jeden den Abstand aus. Ist einer näher als der bisher gemerkte, wird er zum neuen `naechster`. Am Ende steht in `naechster` der Gegner mit dem kleinsten Abstand.

Wichtig: die `isEmpty()`-Prüfung **vorher**, sonst knallt `sensors.others[0]` bei leerer Liste.

Schritt 2 — “wo steht der Gegner relativ zu mir?” (dx / dy)

```
val dx = naechster.position.x - self.x // + = Gegner rechts von mir, - = links
val dy = naechster.position.y - self.y // + = Gegner unter mir, - = über mir
```

Beispiel: ich stehe bei (3,7), Gegner bei (7,8). `dx = 7 - 3 = 4` (Gegner ist 4 nach rechts), `dy = 8 - 7 = 1` (1 nach unten).

Schritt 3 — nur eine Achse pro Tick (die mit dem größeren Abstand) `Action.Move` erlaubt pro Tick **nur eine** Richtung — nicht diagonal. Ihr müsst euch also entscheiden: erst nach links/rechts fliehen oder erst nach oben/unten?

Faustregel: **flieht zuerst auf der Achse, wo der Gegner weiter weg ist.** Warum? Dort bringt ein Schritt den größten Sicherheitsabstand. Vergleicht `abs(dx)` mit `abs(dy)`:

Gegner weit rechts, kaum drüber: `dx = 4, dy = 1`
`abs(dx)=4 > abs(dy)=1` -> auf der x-Achse fliehen

● . . . G Gegner rechts -> ich fliehe nach links (WEST)
←●

Schritt 4 — Vorzeichen umdrehen (DAS ist die Flucht) Beim **Verfolgen** (Epic 2) lauft ihr *in* Richtung `dx/dy`. Beim **Fliehen** genau **andersrum**. Merkt euch das als Spiegel:

Gegner steht ...	<code>dx/dy</code>	Verfolgen (hin)	Fliehen (weg)
rechts von mir	<code>dx > 0</code>	EAST	WEST
links von mir	<code>dx < 0</code>	WEST	EAST
unter mir	<code>dy > 0</code>	SOUTH	NORTH
über mir	<code>dy < 0</code>	NORTH	SOUTH

Als Code (nur der Flucht-Zweig):

```
if (abs(dx) > abs(dy)) {  
    // x-Achse: Gegner rechts (dx>0) -> ich nach WEST, sonst nach EAST  
    return Action.Move(if (dx > 0) Direction.WEST else Direction.EAST)  
} else {  
    // y-Achse: Gegner unten (dy>0) -> ich nach NORTH, sonst nach SOUTH  
    return Action.Move(if (dy > 0) Direction.NORTH else Direction.SOUTH)  
}
```

Denkt an die Karte: y wächst nach **unten**. “Nach oben fliehen” heißt NORTH und macht y **kleiner**. Wer hier y mit “unten = kleiner” verwechselt, baut genau den Vorzeichenfehler ein.

Schritt 5 — der else-Zweig (genug HP) Kriterium 2: Bei `health >= 20` soll der Bot **normal** weitermachen. Was “normal” ist, hängt davon ab, wie weit ihr seid:

- Habt ihr Epic 2 noch nicht: einfach `Action.Move(Direction.entries.random())` oder auf den Gegner **zu** laufen (Flucht-Code mit **nicht** getauschtem Vorzeichen).
- Habt ihr Epic 2 schon: hier eure Angriffs-Logik (schießen / verfolgen) einsetzen.

Häufige Fehler bei genau dieser Story: - **Vorzeichen verdreht** -> Bot rennt beim Fliehen genau **auf** den Gegner zu. Der wichtigste Test: HP künstlich runter (z.B. gegen mehrere Bots), zuschauen — läuft er wirklich **weg**? - **? :-Fallback vergessen** -> Absturz, sobald der letzte Gegner stirbt. Der Bot wird dann von der Engine “eingefroren”. - **Beide Achsen gleichzeitig bewegen wollen** -> geht

nicht, Move ist eine einzige Richtung. Immer nur eine Achse pro Tick. - **health <= 20 statt < 20**
-> Grenzfall; die Story sagt "unter 20", also < 20. Kleinigkeit, aber im Review erwähnen.

Epic 2 – Angriff

Story 2.1 – Dauerfeuer in feste Richtung · 1 Punkt

Ziel: Bot schießt jeden Tick in dieselbe feste Richtung.

Leitfragen: - Welche Action schießt? -> Action.Shoot(Direction.EAST). - Warum trifft ihr damit fast nie? -> Ein Schuss trifft nur, wenn ein Gegner **exakt** in derselben Zeile (bei EAST/WEST) bzw. Spalte (bei NORTH/SOUTH) steht. Zufällig passt das selten.

Taktik-Gedanke: Testet gegen StillstandBot — der bewegt sich nicht, also könnt ihr euch so hinstellen, dass die Schüsse treffen. Diese Story ist nur zum Kennenlernen von Shoot; die “richtige” Zielerei kommt in 2.2.

Story 2.2 – Auf Gegner in Sichtlinie zielen · 3 Punkte

Ziel: Nur schießen, wenn ein Gegner **wirklich** in der Schusslinie steht — und dann in die richtige Richtung.

Was heißt “in Sichtlinie”? Auf dem Raster gibt es keine schrägen Schüsse. Ihr trefft jemanden nur, wenn er in derselben **Spalte** (gleiches x) oder derselben **Zeile** (gleiches y) steht:

G gleiches x wie ● -> Schuss nach NORTH trifft G
 .
 ● · · G gleiches y wie ● -> Schuss nach EAST trifft den rechten G

G kann NICHT getroffen werden, wenn er weder in der Zeile noch in der Spalte steht (schräg):

- • G
- • • weder gleiches x noch gleiches y $\rightarrow$ kein Schuss

Leitfragen: - Wie prüfe ich, ob ein Gegner treffbar ist? -> für jeden Gegner testen: `gegner.position.x == meinX || gegner.position.y == meinY`. - Wie finde ich den nächsten davon? -> alle Gegner mit einer for-Schleife durchgehen, treffbare merken, den mit dem kleinsten Abstand behalten (wie in Story 1.3, nur zusätzlich mit der "in Linie?"-Prüfung). - Wie leite ich aus der Position die Schussrichtung ab? -> gleiches x und Gegner hat kleineres y -> NORTH; größeres y -> SOUTH; gleiches y und größeres x -> EAST; sonst WEST. - Kein Gegner in Linie? -> **nicht** ins Leere schießen, lieber `Action.Wait` (oder in 2.3: hinlaufen).

Häufiger Fehler: Erst den nächsten Gegner suchen und **dann** prüfen, ob er in Linie steht. Besser umgekehrt: **erst filtern** (wer ist überhaupt treffbar), **dann** den nächsten davon nehmen. Sonst zielt ihr auf jemanden, den ihr gar nicht treffen könnt.

Story 2.3 – Gegner verfolgen · 3 Punkte

Ziel: Steht kein Gegner in Schusslinie -> einen Schritt auf den nächsten zulaufen. Kombiniert mit 2.2 ergibt das einen echten Jäger-Bot.

Die Gesamtlogik pro Tick:

```

nächsten Gegner suchen
|
steht er in Linie (gleiches x oder y)?
/      \
ja       nein
|        |
SCHIESSEN  HINLAUFEN
(Richtung aus 2.2) (dx/dy -> Schritt zum Gegner)

```

Leitfragen: - “Hinlaufen” ist dasselbe wie “Fliehen” aus 1.3 — nur mit **richtigem** (nicht umgekehrtem) Vorzeichen. Welche Achse zuerst? -> wieder die mit dem größeren Abstand. - Warum nähert man sich damit “diagonal treppenförmig”? -> weil man abwechselnd ein Feld in x- und dann in y-Richtung geht, bis man in einer Linie steht.

Taktik-Gedanke: Das ist genau der ChaserBot aus bots/examples/. Wenn ihr nicht weiterkommt: **nicht abschreiben**, aber die Struktur anschauen und verstehen, dann selbst nachbauen.

Epic 3 — Strategie & Zustände

Story 3.1 — Zustandsmaschine (Patrouille / Angriff / Flucht) · 5 Punkte

Ziel: Bot hat drei klar benannte Zustände und wechselt je nach Lage.

Das ist keine **neue** Logik — es ist das **Aufräumen** von allem aus Epic 1 & 2 in drei saubere Schubladen. Denkt an eine Ampel: je nach Situation ein anderer Zustand, jeder Zustand macht etwas klar Unterscheidbares.

```

health < 20  —————>  FLUCHT      (Logik aus Story 1.3)
und Gegner da?

Gegner da (aber genug HP) ———>  ANGRIFF   (Logik aus Story 2.2 + 2.3)

kein Gegner —————>  PATROUILLE (Logik aus Story 1.1)

```

Leitfragen: - Wie stelle ich drei Zustände dar? -> am saubersten mit `enum class BotState { PATROUILLE, ANGRIFF, FLUCHT }`. - Wie bestimme ich den Zustand? -> mit einem `when { ... }`, das die aktuelle Lage prüft. **Reihenfolge zählt:** zuerst FLUCHT prüfen, dann ANGRIFF, dann Rest. - Warum zuerst Flucht? -> Ein fast toter Bot soll fliehen, **auch wenn** ein Gegner in Schussweite steht. Prüft ihr Angriff zuerst, stirbt er beim Angreifen.

Taktik-Gedanke: Muss der Zustand gespeichert werden? Nein — es reicht, ihn **jeden Tick neu** aus sensors zu berechnen. Das macht den Bot robust: er reagiert immer auf die aktuelle Lage.

Story 3.2 — Zielpriorisierung bei mehreren Gegnern · 3 Punkte

Ziel: Bei mehreren Gegnern nicht wahllos den erstbesten angreifen, sondern nach einer klaren Regel auswählen.

Der ganze Trick ist ein getauschtes Vergleichs-Kriterium. Bisher habt ihr in der Such-Schleife immer den *nächsten* Gegner gesucht (kleinster Abstand):

```

// nächster Gegner (wie in 1.3 / 2.3)
var ziel = sensors.others[0]
var bester = abs(ziel.position.x - meinX) + abs(ziel.position.y - meinY)
for (gegner in sensors.others) {

```

```

    val abstand = abs(gegner.position.x - meinX) + abs(gegner.position.y - meinY)
    if (abstand < bester) {
        bester = abstand
        ziel = gegner
    }
}

```

Jetzt vergleicht ihr in derselben Schleife einfach ein **anderes Merkmal** — z.B. die HP statt den Abstand:

```

// schwächster Gegner (wenigste HP)
var ziel = sensors.others[0]
var wenigsteHp = ziel.health
for (gegner in sensors.others) {
    if (gegner.health < wenigsteHp) {
        wenigsteHp = gegner.health
        ziel = gegner
    }
}

```

Gleiche Schleifen-Struktur, nur `< bester` wird zu `< wenigsteHp`. Das ist der ganze Lernpunkt der Story.

Leitfragen: - Welche Regel wollt ihr? Mögliche Kriterien: - **schwächster zuerst** (`gegner.health`) — schaltet ihr am schnellsten aus einem Kampf aus. - **nächster zuerst** (Abstand) — am leichtesten zu erreichen/treffen. - Egal welches — schreibt als Kommentar dazu, **warum** ihr es gewählt habt. - Der Rest (schießen/hinlaufen) bleibt exakt wie in Epic 2, nur mit `ziel` statt `nächster`.

Häufiger Fehler: Nur das Kriterium ändern, aber vergessen, danach überhaupt zu schießen/laufen. Denkt dran, die Aktion aus 2.2/2.3 anzuhängen.

Story 3.3 — Kür-Aufgabe (freie Idee) · 5 Punkte

Ziel: Eigene Idee, die über die Vorgaben hinausgeht. Ihr formuliert die Story selbst, der Dozent bestätigt kurz die Machbarkeit.

Was geht gut mit der API? - **An der Wand entlang patrouillieren** (Position mit `arenaWidth/arenaHeight` vergleichen). - **Gegner in die Ecke drängen** (Bewegung relativ zu mehreren `others`). - **“Nur schießen, wenn sicher”** — z.B. nur feuern, wenn genau **ein** Gegner in Linie steht, nicht mehrere. - **Gegner-Zug merken** — mit einer `var`-Property speichert ihr die letzte Position eines Gegners und schätzt, wohin er läuft. (Vorher mit Dozent kurz besprechen.)

Was geht NICHT? - Ihr könnt **nicht in die Zukunft sehen**. `sensors` zeigt nur den Zustand **jetzt**, bevor sich alle bewegen. Ein Schuss auf ein laufendes Ziel ist immer eine **Wette**, kein sicherer Treffer. - Ihr könnt nicht sehen, was ein Gegner als Nächstes *entscheidet* — nur, wo er gerade steht.

Taktik-Gedanke: Fangt klein an. Eine simple, funktionierende Idee ist mehr wert als eine geniale, die nicht läuft. Baut auf eurem Epic-3.1-Bot auf und fügt **einen** cleveren Zusatz hinzu.

Epic 4 — Qualität (optional)

Story 4.1 — Unit-Test für eure Logik · 2 Punkte

Ziel: Ein Test, der `decide()` mit selbst gebautem `Sensors` aufruft und das Ergebnis prüft.

Warum geht das so einfach? `decide()` ist eine **reine Funktion**: rein Sensors, raus Action. Ihr braucht keine laufende App — ihr baut einfach von Hand eine Situation und prüft, was rauskommt.

Leitfragen: - Wie baue ich eine Test-Situation? -> `RobotState(...)` und `Sensors(...)` sind normale data class-Objekte, die ihr direkt erzeugen könnt. - Welche Situation teste ich? -> z.B. Gegner steht östlich in gleicher Zeile -> erwartet: `Action.Shoot(Direction.EAST)`. - Wo muss die Datei liegen? -> `src/test/kotlin/bots/teamX/...` (Ordner ggf. neu anlegen), package muss zum Pfad passen.

Taktik-Gedanke: Testet einen Fall, bei dem ihr die Antwort **sicher** wisst (z.B. Gegner exakt rechts). Dann seht ihr sofort, ob eure Richtungslogik stimmt.

Story 4.2 — Testduell protokollieren · 1 Punkt

Kein Code. App starten, euren Bot + einen Beispiel-Bot wählen, Match laufen lassen, Ergebnis in 1-2 Sätzen notieren.

Epic 5 — Turniervorbereitung

Story 5.1 — Finale Version festlegen · 1 Punkt

Stellt sicher, dass in `teamXBots = listOf(...)` **genau** die Bot-Version steht, die antreten soll — nicht drei halbfertige Zwischenstände. `./gradlew build` muss grün sein.

Story 5.2 — Strategie in 1-2 Sätzen · 1 Punkt

Könnt ihr in eigenen Worten sagen, **wann** euer Bot angreift, flieht, patrouilliert? Wenn ja: fertig. Wenn nicht: das ist ein Zeichen, dass euer Bot vielleicht noch zu unübersichtlich ist — gute Gelegenheit zum Aufräumen.

Wenn ihr komplett feststeckt

1. **Läuft der Bot überhaupt?** Zurück zu Story 1.1 — taucht er in der Auswahl auf, bewegt er sich?
2. **Absturz?** Meist `sensors.others` leer und trotzdem `sensors.others[0]` oder `.first()` benutzt. Immer **vorher** mit `if (sensors.others.isEmpty()) return Action.Wait` abfangen.
3. **Läuft/schießt falsch herum?** Vorzeichen von `dx/dy` prüfen und dran denken: `y` wächst nach unten.
4. **Schießt nie?** Steht ein Gegner wirklich in gleicher Zeile/Spalte? Testet erst gegen `StillstandBot`, da ist es kontrollierbar.
5. Schaut euch die fertigen Bots in `bots/examples/` an — **nicht abschreiben**, sondern verstehen und selbst nachbauen.